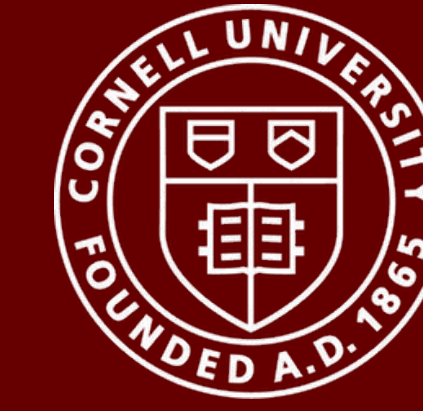


Graph Attention Networks

Anish Bhupalam, Anagha A. Ram, Jason Nair
Cornell University



Cornell
Engineering

Introduction & Summary

- *Graph Attention Networks* by Petar Velickovic et al. introduce attention-based neural networks that **learn importance of neighbors** in graph data.¹
- Demonstrate strong performance on citation networks like Cora & PPI benchmarks, **rivaling and often surpassing** prior graph neural network models (Graph Convolutional Networks (GCN) & GraphSAGE).
- Useful for learning from **naturally relational data** such as social networks, molecules, or citation graphs.

- We use Graph Attention Networks (GATs), a graph neural network that uses a learned attention mechanism to assign **different levels of importance** to each neighbor node when aggregating features. As a baseline, we use Const-GAT, which uses fixed attention weights so that **each neighbor is treated equally**.
- We aimed to reproduce the inductive node classification experiment on a Protein-Protein Interaction dataset. We **evaluate** using the **F1 score**, which balances precision and recall. Our target was a averaged F1 score of 0.973 ± 0.01 for GAT and 0.934 ± 0.01 for Const-GAT.¹

- Our motivation in choosing this paper is to study an elegant and adaptive graph learning approach, comparing GAT with Const-GAT to understand how learned attention **reduces the impact of noisy** or less informative connections & **improves generalization to unseen data**.

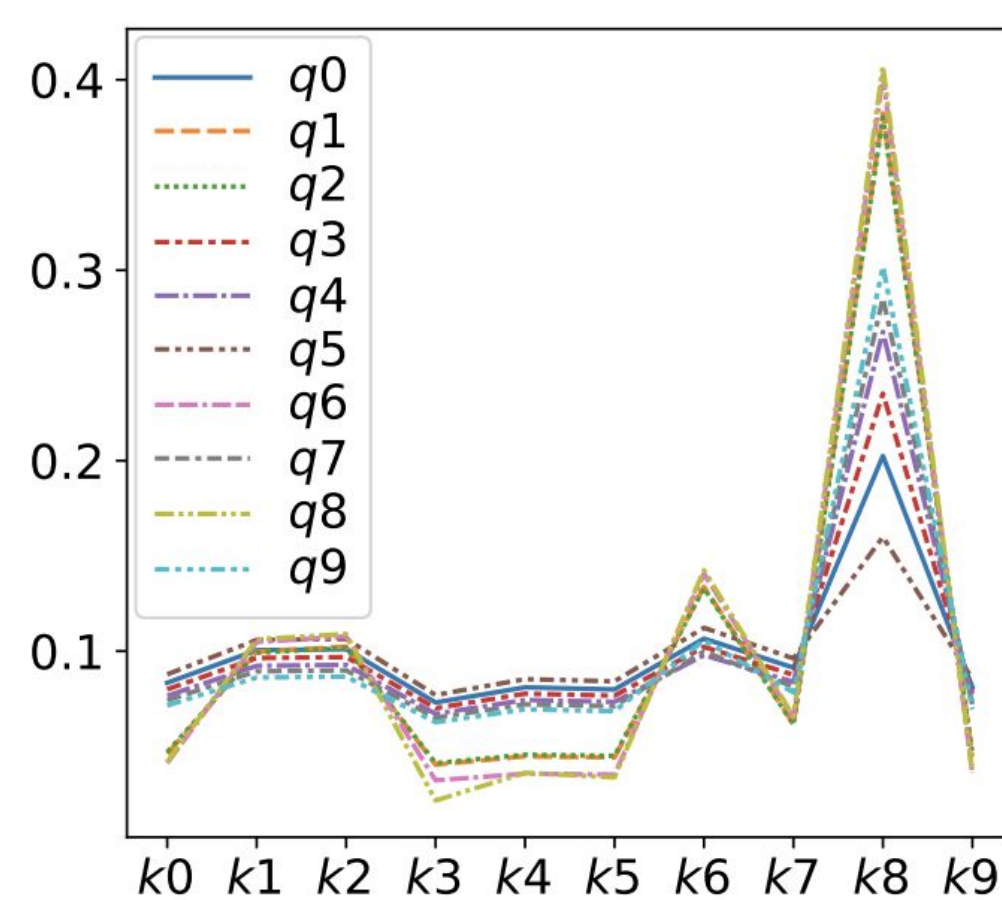


Fig. 1. Attention weights in GAT showing constrained variation across queries, limiting improvement across uniform Const-GAT.²

Methodology

- We implemented GAT in **PyTorch Geometric** on Google Colab, the current standard for graph-based deep learning rather than the original TensorFlow implementation. The core building block is a graph

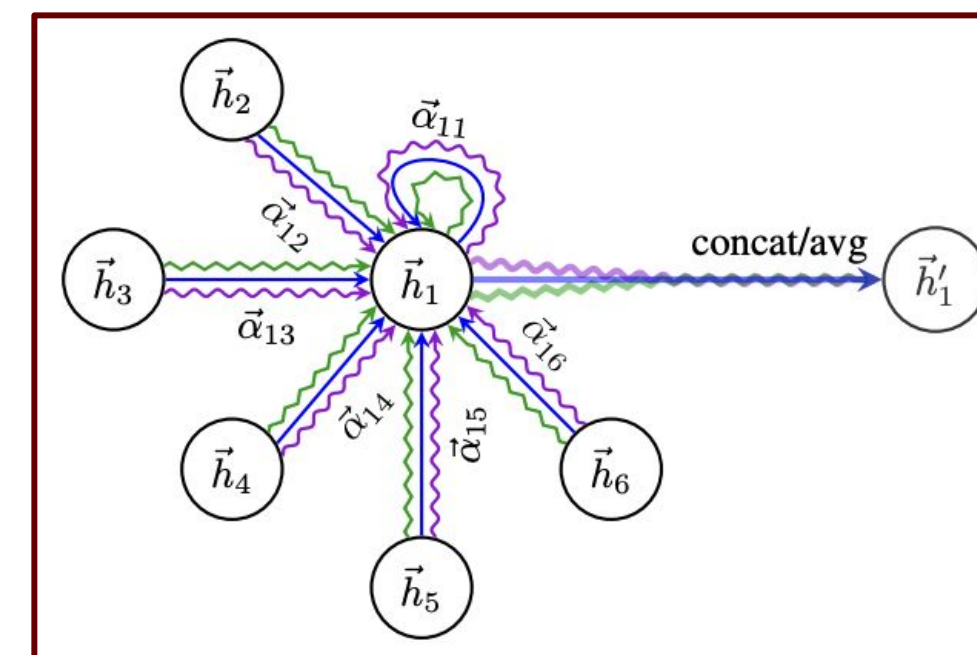


Fig. 2 Multi-head GAT: node i aggregates neighbor features via attention & combines heads to form h_i' .

attentional layer where each node aggregates features from its first-order neighbors using **learned, normalized attention** coefficients. For each edge, we compute the raw attention score followed by a LeakyReLU activation and normalize via softmax over **each node's neighborhood** to produce a weighted sum of features.

- GAT has a three-layer architecture. The first two layers each use 4 attention heads computing 256 features each, followed by ELU activations and skip connections. The final layer uses 6 attention heads each computing 121 features, averaged and passed through a logistic sigmoid. Const-GAT **only differs** in that it **fixes all attention weights** uniformly to 1 rather than learning them.¹

- We trained with Adam at a learning rate of 0.005, batch size of 2 graphs, and early stopping with patience of 100 epochs on validation micro-F1. We also applied **gradient clipping**: we noticed that training occasionally became unstable with exploding gradients.

- The PPI dataset is loadable directly via `torch_geometric.datasets.PPI`. On Google Colab's T4 GPU, each training run took **~20 minutes** given the large hidden dimension (1024 features/node) and **full-neighborhood aggregation** (fig. 1). Running 10 runs each for GAT and Const-GAT gives a total of **20 runs**, or approximately **7 hours of GPU time**.

Results

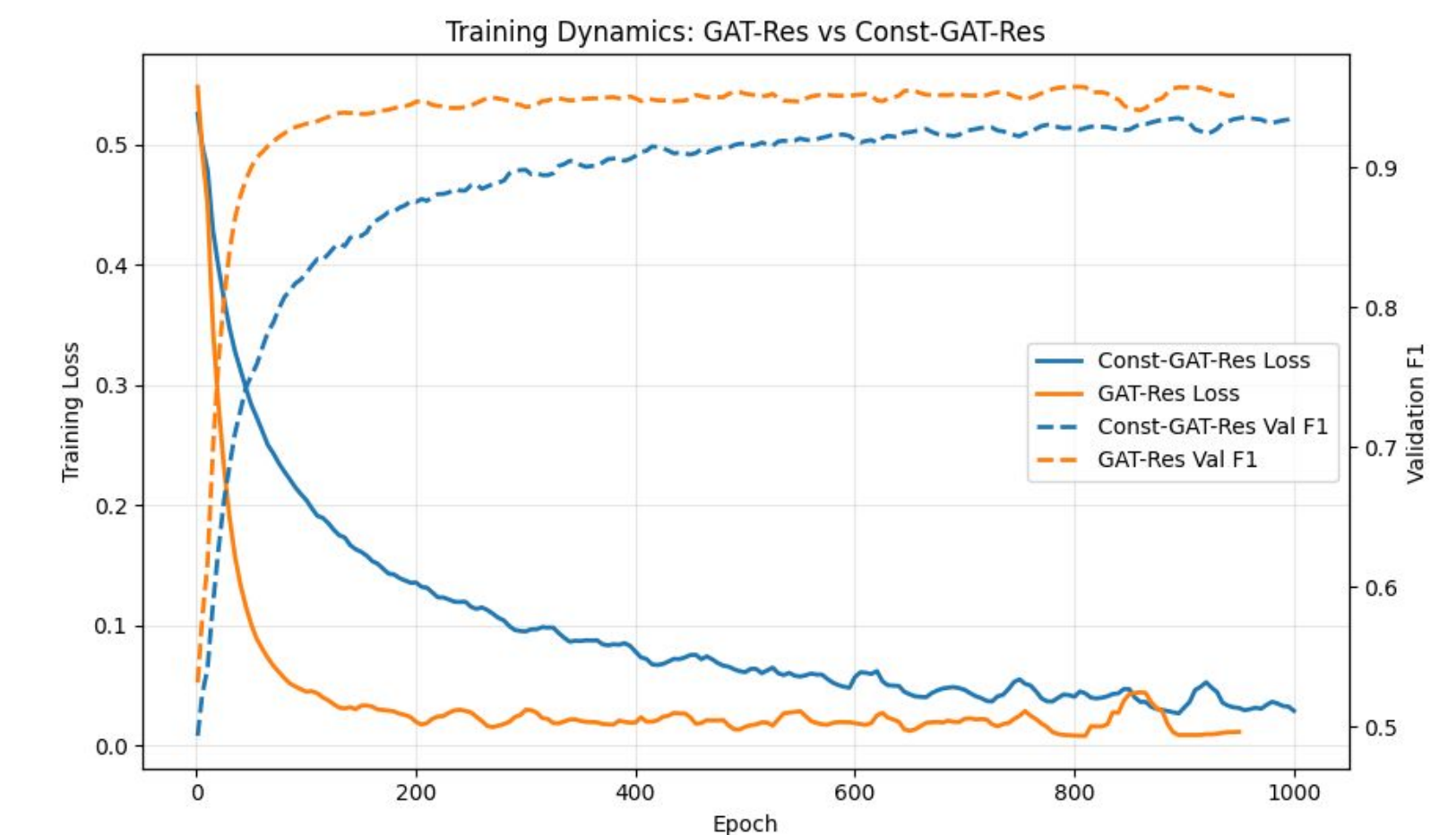


Fig. 3 Training curves comparing GAT-Res & Const-GAT-Res, showing faster convergence and higher validation F1 for GAT-Res

Method	PPI Test F1
Random	0.396
MLP	0.422
GraphSAGE*	0.768
Const-GAT (PAPER)	0.934 ± 0.006
GAT (PAPER)	0.973 ± 0.002
Const-GAT-Res (OURS)	0.9574 ± 0.0004
GAT-Res (OURS)	0.9719 ± 0.0024

Fig. 4 Replication of summary results of micro-averaged F1 scores for PPI dataset, based on Table 3 from *Graph Attention Networks* by Petar Velickovic et al.

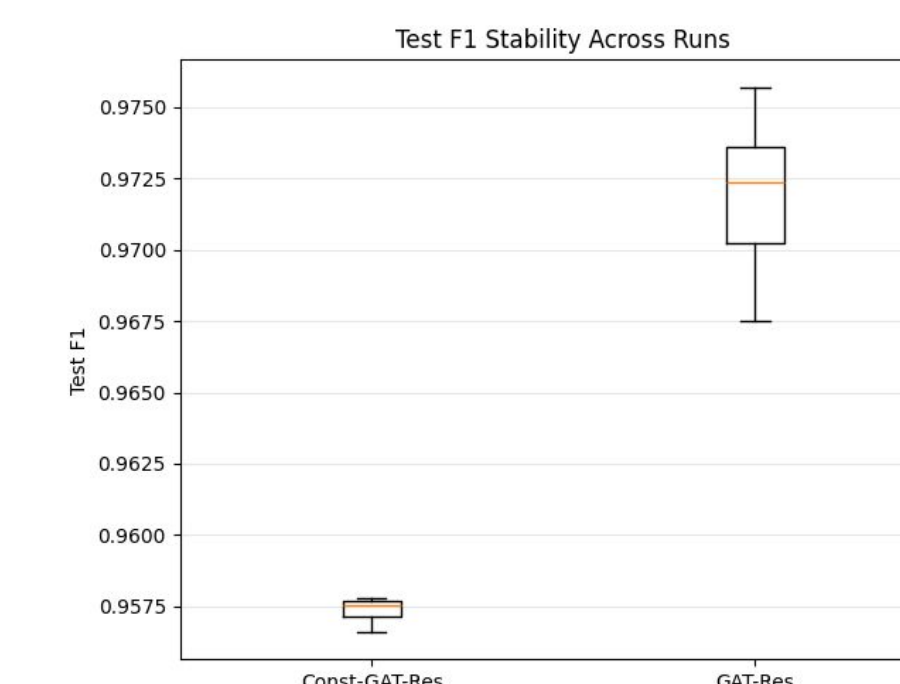


Fig. 5 Test F1 stability across runs showing higher performance and variability for GAT-Res compared to Const-GAT-Res

Our implementation produced a F1 score of **0.9719 for GAT** and 0.9574 for Const-GAT, exceeding the paper's results. This was due to us adding **per-head residual connections**, which increased F1 by ~0.01. The box plot shows us that Const-GAT had very low variance in F1 score across runs, but the **attention mechanism in GAT** caused a higher variance.

Conclusion

Our reproduction confirms that graph attention networks achieve strong node classification performance across benchmarks, due to the combined effect of **full neighborhood aggregation** and **learned attention weighting** of neighbor nodes.